全国高校 AI Web 开发挑战赛 · 技术文档

守望——基于多智能体编排的安全运营平台

AI Native 安全运营 · 多智能体编排 · 治理优先的 Agentic SOC

项目	内容
参赛队伍	（团队信息：队员 1-3 人，报名后按组委会要求填写）
所属学校	湖南科技大学
参赛组别	B 组（本科）
指导教师	（按报名信息填写）
文档版本	v1.0（2026-11-09，数字对账日）
代码仓库	github.com/x2789279228-droid/security-of-agent-max

2026 年 11 月

目　录

一、项目概述.....1

二、需求分析.....2

三、系统架构设计.....3

四、AI 技术方案（选型 · 集成 · 优化）.....5

五、核心功能实现.....8

六、测试与部署.....10

七、技术创新点.....11

八、应用创新点.....12

九、AI 辅助开发工作流披露.....13

十、总结与展望.....13

一、项目概述

「守望」是一套 AI Native 的安全运营平台（SOC）：以大模型多智能体为核心

安全运营（SOC）的行业现状是「告警过载 + 人才稀缺」：一名值班分析师每班需处理数百条告警，其中超过九成是低价值重复；而引入 LLM 直接自动研判又带来新风险——模型幻觉、工具滥用、不可解释的自动处置。守望的回答是「治理优先的 Agentic SOC」：让 LLM 真正参与规划、研判与问答，但每一步都受程序化否决闸约束，拒绝必须给出可审计的理由，不确定必须显式弃权（abstain）。

平台以现代 Web 形态交付：React 19 前端提供对话式 AI Copilot、思维链可视化审计、实时事件大屏；FastAPI 后端提供 222 个 REST 端点与标准 MCP 协议端点；Flink/Kafka 承担流式解耦；全部 AI 决策可回放、可评测、可对账。

维度	实测数字（2026-11-09 对账）
后端测试	3283 项（289 个文件，含 Kafka+PG testcontainers 集成测试）
检测规则	1675 条 Sigma（20 自研 + 1641 SigmaHQ + 手适配），184 条经 FP=0 门禁转正
REST 端点	222 paths / 19 routers（openapi 3.1 实测口径）
MCP 协议	标准 JSON-RPC 2.0 端点（initialize/tools/list/tools/call），15 项协议测试
生成侧忠实度	faithfulness 0.9875 / abstain 5%（20 查询真实 LLM 实测）
检索基线	词法层 MRR 0.9917 / recall@5 1.0（60 语料回归基线，提交入库可复现）
前端	18 条懒加载路由 / 65 组件 / TypeScript strict 全开 / 37 项逻辑测试

CI 门禁	13 个 job：测试覆盖率 55% 门禁、API 契约 diff、密钥泄漏扫描、镜像构建

二、需求分析

2.1 场景与用户

目标用户是高校与企业安全运营中心的值班分析师与安全工程师。典型场景：值班期间面对连续涌入的告警，需要快速判断「这是不是真实攻击」、「影响面多大」、「现在该做什么」；新人还需要理解判断依据以完成能力成长。

2.2 核心痛点与需求映射

痛点	需求	平台能力
告警过载，人工分诊慢	自动分级分诊 + 可解释研判	四层 Agent 审计管线 + 思维链 DAG 可视化
LLM 幻觉不可用	结论必须锚定证据	七层 grounding + faithfulness 闸（0.75 阈值）
AI 自动处置危险	动作可控可审计	MCP 治理网关四层检查 + 审批队列 + 默认 dry_run
知识散落难复用	领域知识问答可溯源	RAG 混合检索 + 引用编号 + 生成侧忠实度评测
新人成长周期长	决策过程可回放学习	全链路轨迹留痕 + 复盘报告 + 红蓝自博弈训练场
传统 SOC 封闭难集成	标准协议接入	MCP JSON-RPC 端点 + Kafka 生态 + Sigma 规则体系

需求优先级遵循「可信 > 可用 > 好看」：一个敢把「无法根据证据回答」写在脸上的系统，比一个永远给出漂亮答案的系统更接近生产可用。

三、系统架构设计

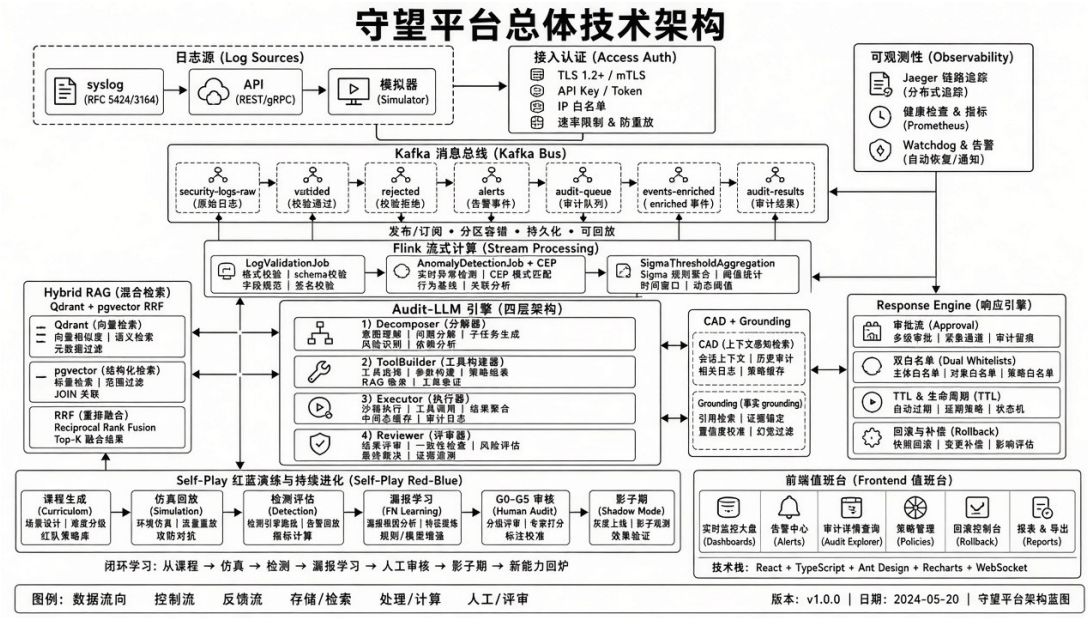


图 3-1 平台五层总体架构

3.1 分层职责

层	职责	关键技术
接入层	日志/端点/流量/情报多源接入	Sysmon/EVTX 解析、Suricata、MISP/KEV 情报
流处理层	解耦、削峰、窗口计算	Kafka 16 Topic、Flink 6 作业
检测层	规则+异常+关联多引擎	Sigma 1675 条、五维异常检测、跨源关联
智能体层	规划/研判/复核/治理	四层 Agent 管线、Temporal 编排、MCP 治理
交互层	值班工作台与 AI 对话	React 19、SSE 流式、思维链 DAG

3.2 技术栈选型（前端为核心交付面）

选型	理由

React 19 + Vite 8/Rolldown	并发特性成熟；Rolldown 构建快且产物干净，18 条路由全部 React.lazy 按需分包
TypeScript strict 全开	编译期拦截空值与隐式 any；配合 openapi-typescript 生成类型与后端同源
Zustand（3 store）	只管登录态/联动/服务健康三类真全局态；服务端数据不进全局 store，避免状态泥球
手写 POST-SSE 解析器	EventSource 不支持 POST 与大载荷；自实现跨 chunk/CRLF/多 data 帧解析，8 条单测钉住
FastAPI + Pydantic v2	运行时校验与 openapi 契约一体；JSON 输出靠 schema 收敛，越界值程序化修正
PostgreSQL + Qdrant + Redis	关系/向量/缓存各司其职，均可一条 compose 起栈

3.3 数据流与事件生命周期

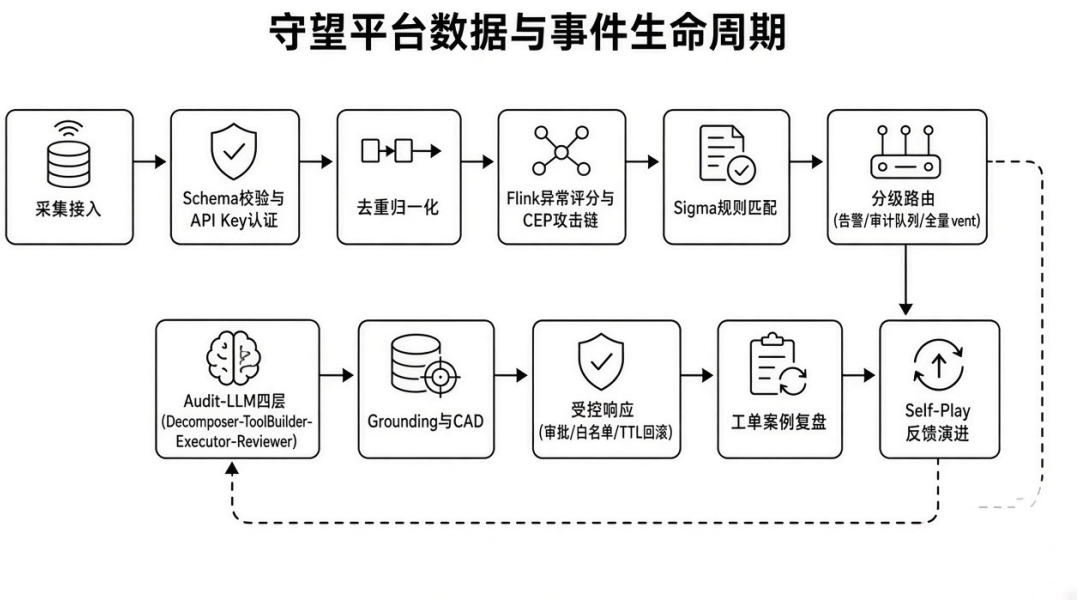


图 3-2 事件从接入到闭环的生命周期

四、AI 技术方案（选型 · 集成 · 优化）

4.1 大模型集成

选型：LLM 网关采用 OpenAI 兼容协议接入 MiniMax-M3 系列。理由有三：① 协议兼容使模型可替换——评测脚本能对任意兼容网关复跑；② MiniMax-M3 支持 reasoning 拆分（思考独立字段），流式输出不污染 JSON 正文；③ 阶段性 token 成本可控。Prompt 全部入库 prompts/*.j2 模板目录（Jinja2），由仓库守卫强制（.py 内不允许出现内联 system prompt），版本化可审计。

输出稳定可控的四道防线：① 提示词内嵌 JSON Schema 约束与「无证据自动丢弃」条款；② 解析层 extract_json 容错抽取；③ Pydantic 校验，越界值程序化收敛（如严重度枚举），校验失败标记 schema_invalid 强制人审；④ 降级链——预算耗尽、无 key、上游故障分别返回显式 fallback 标记，绝不把失败伪装成结论。

4.2 Agent 工作流：四层管线与真决策规划

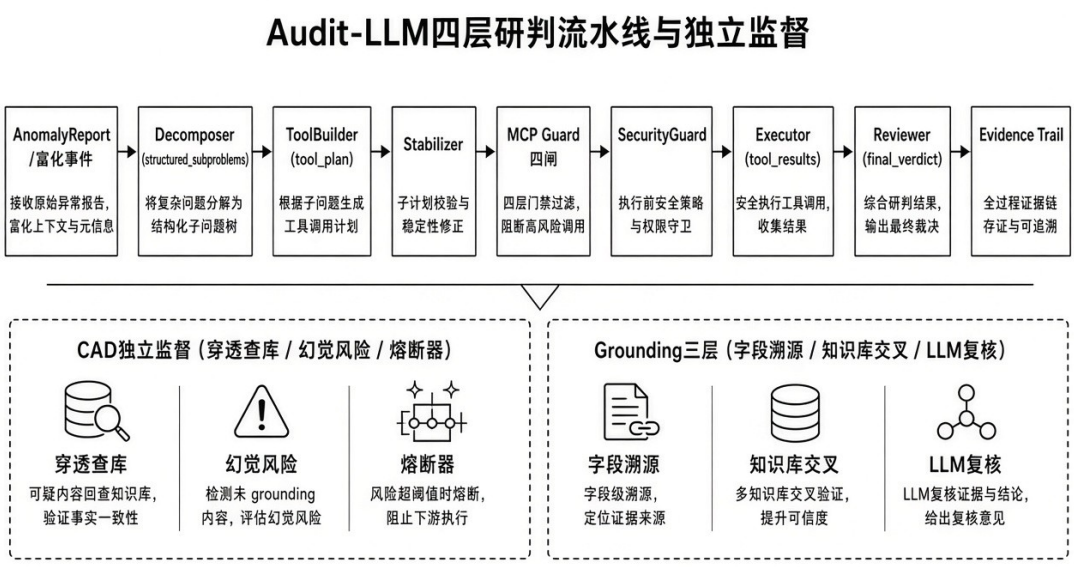


图 4-1 四层 Agent 审计管线

管线为 Decomposer → ToolBuilder → Executor → Reviewer 四层，接入生产入口（log_ingestion），并行 SubAuditor 分块研判，Temporal 保证长任务可靠性。本次参赛完成的关键升级——Decomposer LLM 真决策：深度审核档由 LLM 规划器从 7 项取证能力（24h 事件回查/IP 基线/目标事件/攻击链/实体关联/记忆检索/知识检索）中挑选实际需要的子集，输出经 Pydantic 校验并按防幻觉白名单过滤；「综合分析」与「复核」两个安全底线由代码强制保留，LLM 只能裁剪取证面、不能拆底线。规划失败/超时/全幻觉时回退全量规则模板（plan_source=rules_fallback），规划溯源写入思维链（plan_source/plan_selected），前端 SecurityAudit 页可见。

Reviewer 的设计体现「只准降级」原则：LLM 复核不得把结论改得更严重，只能维持或降级，升级必须由非 LLM 信号触发——这是一个可被程序化验证的安全性质。

4.3 RAG：混合检索与双向评测

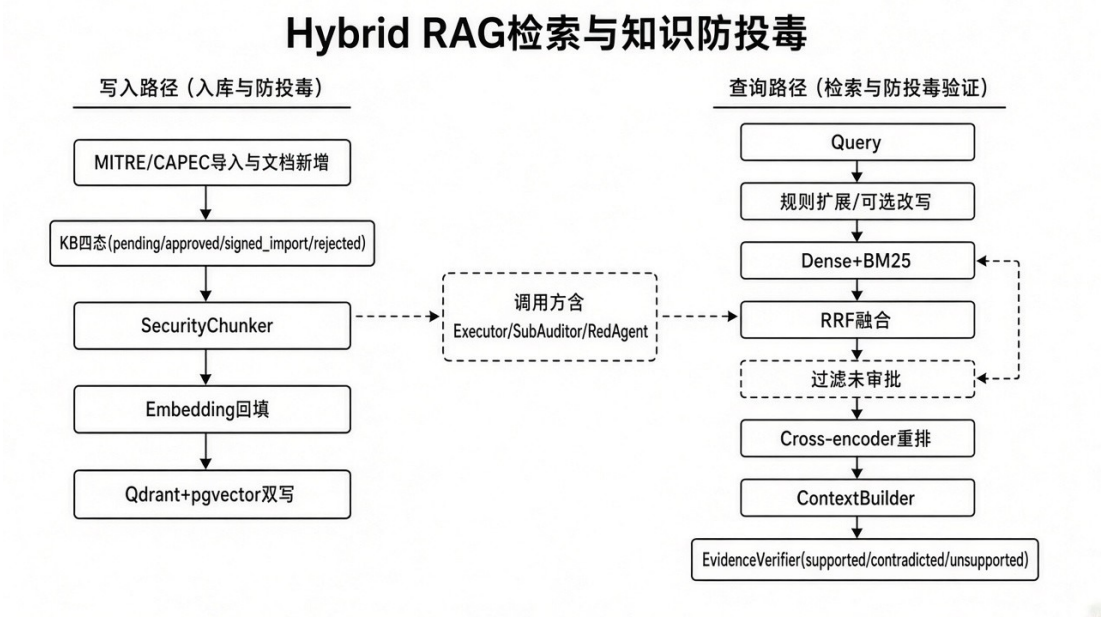


图 4-2 RAG 混合检索管线

检索链：规则改写/关键词扩展 → dense（Qdrant）+ sparse（BM25）双路召回 → RRF 融合 → 交叉编码器重排（可选 Infinity 本地服务）→ HyDE 空结果重试。引用以编号形式回传前端，用户可逐条溯源到知识块。

评测双向可复现：① 检索侧——60 语料/60 查询提交入库，词法层基线 MRR 0.9917（定位为回归基线而非能力上限）；hybrid 模式起真实栈以 rag.retriever 实跑同黄金集，前置检查不满足则如实退出并给 runbook，不伪造数字。② 生成侧（本次新增）——BM25 top5 证据 → 真实 LLM「仅凭证据作答」→ 以生产同款 faithfulness_gate 逐断言评分：实测 faithfulness 0.9875、abstain 率 5%、1 条无证据主张被识别。这回答了评分细则的核心问题——「检索增强是否显著提升生成质量」有了带失败样本的可对账数字。

4.4 MCP 协议集成（本次新增）

在既有治理网关之上补齐标准 MCP 协议面：POST /api/mcp 提供 JSON-RPC 2.0 的 initialize / notifications/initialized / ping / tools/list / tools/call。协议层只做翻译、不放松治理：协议错误（未知方法/坏参数）映射 -32601/-32602；治理拒绝（RBAC/策略/审批挂起）映射为 MCP 工具级 isError 结果，决策与四层检查明细放进 structuredContent.guard 透出给客户端。HTTP JWT 角色映射 guard RBAC，客户端无法通过参数自报角色提权。15 项协议测试覆盖形状、错误码、通知语义、批量请求与治理不放松。

4.5 多模态与诚实降级

Copilot 支持单图研判：粘贴/上传截图（PNG/JPEG/WebP/GIF ≤6MB）转 data URL，经 OpenAI 兼容 image_url 内容块进入视觉链路。视觉模型未配置时 UI 如实置灰并说明原因，后端返回 vision_note 警告条——「不伪装有能力」本身是产品语义。

4.6 AI 安全与可控性（最强项）



图 4-3 七层 grounding 与治理闸

七层防幻觉：未信日志包裹（<untrusted_log>，日志不是指令）→ 输入清洗 → 结构化 grounding 校验（断言必须绑定事件 ID 与逐字证据）→ 知识库交叉验证 → faithfulness 闸（阈值 0.75，逐断言评估不过即拦）→ veto_gates 程序化否决（confirmed 必须绑定非 LLM 强信号）→ 审批队列（高危动作人工确认）。叠加 LLM 日预算、限流、并发闸与 CAD 熔断，构成完整的成本与故障兜底。

五、核心功能实现

5.1 前端 AI 交互（评分重点）

Copilot 对话页：手写 POST-SSE 解析器（跨 chunk 边界逐字节安全、CRLF 规范、多 data 行合并、心跳注释跳过、尾帧宽容冲洗）驱动流式渲染；9 类帧归约为纯函数（可单测）；ThinkingRail 三态思考轨（执行中/完成/跳过——跳过即降级，如实标注）；引用 chips 溯源；停止（AbortController）与失败重试；

每条消息独立 ErrorBoundary。本次优化：token 增量帧 60ms 合流（长会话不再整树重渲染）、历史气泡 memo 化、重试不再丢失附件截图。

5.2 思维链可视化审计

SecurityAudit 页呈现 Decomposer→ToolBuilder→Executor→Reviewer 思维链 DAG，证据链断言↔事件 ID 可点对点核对，LLM 规划溯源（谁定的计划、挑了哪些取证）随步骤 metrics 透出——把「AI 怎么想的」变成值班可核对的工件。

5.3 检测引擎

Sigma 1675 条规则动态管理（影子模式批量导入 ＋FP=0 双门禁转正，184 条执法）；EVTX/Sysmon 解析实测 364 条全量 100%；CSIC-2010 6.1 万条全量回放 FP=0；五维异常检测显式区分「真实阴性/不可判定/失败」，不以精度换假象。

5.4 响应引擎与红蓝自博弈

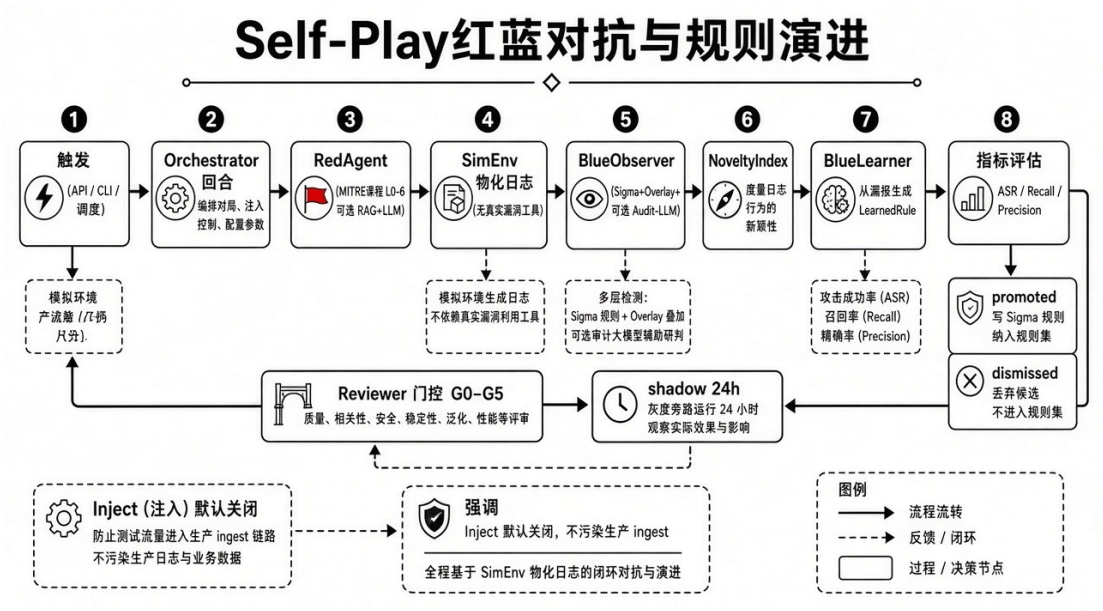


图 5-1 红蓝自博弈训练闭环

响应三态（dry_run/mock/live）默认 dry_run，真实执行需审批与凭据；红队生成对抗样本、蓝队学习、Agent 审计 Agent 的自博弈闭环提供持续训练场。

六、测试与部署



图 6-1 分层测试体系

6.1 测试体系

3283 项后端测试（289 文件）：单元/管线/守卫矩阵/集成（testcontainers 真 Kafka+PG）/评测脚本；37 项前端逻辑测试（SSE 解析器、帧归约、结构断言防回归）。CI 13 个 job 含覆盖率 55% 门禁、openapi→TS 契约 diff、前后端路由契约守卫、密钥泄漏扫描、README 数字对账门禁（防文档失真）、镜像构建。

6.2 评测基准（全部可复现，命令入库）

评测	结果	口径
CSIC-2010 全量回放	FP=0/36000 门禁	公开数据集，逐层消融对照

EVTX-ATTACK 解析	364/364 = 100%	公开攻击样本集
RAG 词法检索	MRR 0.9917 / recall@5 1.0	60 语料回归基线（自建，提交入库）
RAG 生成忠实度	0.9875 / abstain 5%	真实 LLM + 生产同款闸，逐断言评分
LLM 容错矩阵	429/超时/坏 key/断网/5xx 全降级路径	故障注入测试
压测	24,447 条零丢失	Kafka 全链路

6.3 部署

docker compose 一条命令起栈（22 容器含可选 profile）；.env.example 200 键分层注释；fail-closed 设计——必填口令缺失直接报错拒绝启动；演示有六幕剧本 + 一键总控脚本（登录→灌种子→起攻击流量），断网有 rehearsal 兜底。

七、技术创新点

· 治理优先的 Agentic SOC 架构

业界 Agentic SOC 演进多走「能力优先」；守望反其道——先建七层 grounding 与程序化否决闸，再放 LLM 决策权。Reviewer「只准降级」、veto 依赖非 LLM 强信号，都是可程序化验证的安全性质。

· MCP 协议与治理语义的形式化映射

协议错误与治理拒绝在 JSON-RPC error 与 isError 工具结果之间严格分层，拒绝理由全量透出——把「MCP 工具调用安全」从口号变成可测试的协议契约（15 项测试）。

· LLM 规划的防幻觉白名单机制

Decomposer 让 LLM 真正决定取证计划，但选择域收敛在代码定义的能力键白名单内，底线任务代码强制保留——「自主性」与「可预测性」的边界由类型系统与运行时双重保证。

· 生成侧忠实度评测闭环

用生产同款 faithfulness_gate 评测 RAG 生成（而非通用 RAGAS），评测即回归——线上换模型/改提示词必须过同一把尺子，且数字带失败样本可对账。

· 诚实降级作为产品语义

视觉未配置置灰、证据不足明说、预算耗尽显式 fallback、abstain 独立计数——「承认做不到」贯穿前后端，是负责任 AI 的工程化表达。

· 前端工程的可验证性实践

手写 SSE 解析器 + 帧归约纯函数化 + 结构断言测试（对源码做防回归断言）；openapi 契约 diff 与前后端路由契约守卫进 CI；README 数字对账门禁防文档失真。

八、应用创新点

· 值班副驾驶范式

从「看板被动刷新」到「对话主动研判」：任意告警可一键追问 AI，回答锚定证据并可溯源——把 LLM 放在分析师手边而非替代分析师。

· 可解释告警与教学价值

思维链 DAG 让每个告警的判断过程可回放，红蓝自博弈提供攻防训练场——可直接用于高校网络安全实训课程。

· **真实数据集回放的可信演示**

演示不是造数：CSIC/EVTX/CIC-IDS 公开数据集回放 + 真实 LLM 网关 + 失败样本公开呈现，评审可按入库命令逐条复现。

九、AI 辅助开发工作流披露

本项目的开发过程大量使用 LLM 辅助编码：需求拆解、代码生成、测试编写、文档撰写均有人机协作痕迹（Git 历史中存在短间隔批量提交）。我们选择如实披露而非掩饰：① 所有 AI 产出一律经过人工评审与运行验证后合入；② 关键安全组件（治理闸、grounding、MCP 协议层）的每一行均有对应测试钉住；③ 本文档全部数字来自 2026-11-09 实测对账，可由仓库内命令复现。我们认为「会用 AI 开发」与「理解所开发的系统」并不矛盾——3283 项测试与七层治理正是这种理解的证据。

十、总结与展望

10.1 已知限制（诚实清单）

限制	现状与计划
多模态广度	仅图像单模态且视觉模型未配置时走诚实降级；语音/视频未实现
连接器生态	对比商业 SOC 落后 1-2 个数量级，当前聚焦 Sysmon/Suricata/MISP/KEV
RAG 语料规模	60 条评测语料为回归基线；更大规模语料与 hybrid 实跑数字待栈就绪后补
响应执行	默认 dry_run，真实执行需凭据与审批（演示环境为模拟执行器）

10.2 展望

短期：后端 response_model 声明全覆盖以打通前端响应类型；hybrid 检索评测实跑；视觉模型接入。中期：多租户与连接器市场；评测语料扩至千级并引入 LLM-as-judge 交叉验证。长期：以「治理优先」架构参与开源社区，推动 Agentic SOC 的安全基线标准化。